



UNIVERSITI TEKNOLOGI MALAYSIA

SECP2753

DATA MINING

PROJECT 1

Title: Customer Personality Analysis Using Unsupervised Learning

PROF. MADYA DR. ROLIANA BINTI IBRAHIM

NO	NAME	MATRIC NO
1.	CHAU YING JIA	A23CS0213
2.	GUI KAH SIN	A23CS0080
3.	SABRINA HENG WEI QI	A23CS0265

Table of Content

1.0 Introduction	3
2.0 Business and Data Understanding.....	3
2.1 Problem Formulation	3
2.2 Dataset Overview	4
3.0 Data Preparation and Preprocessing.....	6
3.1 Overview and Importance of Data Preparation	6
3.2 Data Preparation Steps in Python	6
3.3 Results of Data Preparation	9
4.0 Model Development	10
4.1 Overview of Unsupervised Learning Methods	10
5.0 Model Performance and Evaluation	14
5.1 Evaluation Metrics for Unsupervised Learning	14
5.2 Analyze Performance	14
5.3 Final Evaluation Summary	16
6.0 Conclusion.....	16

1.0 Introduction

Customer Personality Analysis is a comprehensive approach to understanding the characteristics and behaviours of a company's ideal customers. By analysing customer demographics, purchasing patterns, and engagement history, business can tailor its products, marketing strategies, and services to meet the specific needs and preferences of different customer groups.

In this project, we will explore customer personality data using unsupervised machine learning techniques, focusing on clustering methods to uncover hidden patterns and similar groupings within the data. The dataset contains a variety of features related to customer demographics, purchasing behaviours, campaign responses, and locations of purchases done.

The primary goal is to identify distinct customer segments without prior labels, enabling the company to further comprehend its customer base and adopt more tailored marketing strategies. To achieve this, we apply two clustering algorithms which are K-Means and Hierarchical Clustering. K-Means partitions customers based on similarity in numeric features, while Hierarchical Clustering builds a tree-like structure of customer relationships.

Customer segmentation is crucial for effective decision-making in marketing and product development. This is because understanding which groups are more responsive to online campaigns and which segments have higher spending on luxury products, can help business allocate resources more efficiently and accurately.

By comparing these two clustering results and evaluating the cohesiveness and separability of customer segments, this project aims to assist in developing targeted, data-driven business strategies that enhance customer satisfaction and company performance.

2.0 Business and Data Understanding

2.1 Problem Formulation

Businesses often face challenges when marketing new products or campaigns to a broad customer base. A one-size-fits-all approach to promotion can result in low engagement and wasted marketing resources. Hence, without a clear understanding of the differences among customers, business may fail to deliver the right message to the right audience and becomes challenging to identify meaningful customer segments for effective marketing.

This project aims to group customers based on their demographic and behavioural data using unsupervised learning methods, allowing businesses to better understand their customers and design preferences strategies for each segment.

2.2 Dataset Overview

The Customer Personality Analysis dataset contains of four main categories of detailed information about customers which are their demographics, purchase behaviour, interactions through marketing campaigns, and purchase channels. There is no target variable in this dataset, as the project involves unsupervised learning. Instead of predicting a known outcome, the objective is to identify natural groupings or clusters of customers based on their similarities. Besides, it is a combination of numerical and categorical variables.

Number of attributes: 29

Number of instances: 2240

However, there are two attributes that maintain constant values: Z_CostContact, which has a value of 3, and Z_Revenue, which has a value of 11. Consequently, these attributes function as dummy variables that do not provide any valuable information for analysis or modelling.

Attribute Summary

Attribute Name	Description	Type
ID	Customer's unique identifier	Numerical
Year_Birth	Customer's birth year	Numerical
Education	Customer's education level (Graduation, PhD, Master, Basic, 2nd Cycle)	Categorical
Marital_Status	Customer's marital status (Single, Together, Married, Divorced)	Categorical
Income	Customer's yearly household income	Numerical
Kidhome	Number of children in customer's household	Numerical
Teenhome	Number of teenagers in customer's household	Numerical
Dt_Customer	Date of customer's enrollment with the company	Date
Recency	Number of days since customer's last purchase	Numerical
Complain	1 if the customer complained in the last 2 years, 0 otherwise	Numerical (binary)
MntWines	Amount spent on wine in last 2 years	Numerical
MntFruits	Amount spent on fruits in last 2 years	Numerical
MntMeatProducts	Amount spent on meat in last 2 years	Numerical
MntFishProducts	Amount spent on fish in last 2 years	Numerical
MntSweetProducts	Amount spent on sweets in last 2 years	Numerical
MntGoldProds	Amount spent on gold in last 2 years	Numerical

NumDealsPurchases	Number of purchases made with a discount	Numerical
AcceptedCmp1	1 if customer accepted the offer in the 1st campaign, 0 otherwise	Numerical (binary)
AcceptedCmp2	1 if customer accepted the offer in the 2nd campaign, 0 otherwise	Numerical (binary)
AcceptedCmp3	1 if customer accepted the offer in the 3rd campaign, 0 otherwise	Numerical (binary)
AcceptedCmp4	1 if customer accepted the offer in the 4th campaign, 0 otherwise	Numerical (binary)
AcceptedCmp5	1 if customer accepted the offer in the 5th campaign, 0 otherwise	Numerical (binary)
Response	1 if customer accepted the offer in the last campaign, 0 otherwise	Numerical (binary)
NumWebPurchases	Number of purchases made through the company's website	Numerical
NumCatalogPurchases	Number of purchases made using a catalogue	Numerical
NumStorePurchases	Number of purchases made directly in stores	Numerical
NumWebVisitsMonth	Number of visits to company's website in the last month	Numerical

3.0 Data Preparation and Preprocessing

3.1 Overview and Importance of Data Preparation

Data preparation is essential in the machine learning workflow. Raw data frequently includes inconsistencies, missing values, or non-numeric entries that need to be addressed prior in implementing any algorithms. In this project, our goal is to identify distinct customer segments based on demographic, behavioural, and spending products using unsupervised learning techniques. These data are used to understand customer behaviour, personalize marketing strategies, and improve customer targeting. Therefore, the dataset must be cleaned, encoded, normalized, and divided into training and testing sets to guarantee the best possible model performance.

3.2 Data Preparation Steps in Python

1. Import libraries and Load Dataset

```
import pandas as pd
import numpy as np

# Load the dataset
data = pd.read_csv("marketing_campaign.csv", sep="\t")

# Check data shapes
print("Dataset shape: ", data.shape)

# Preview dataset of the first 5 rows
data.head()
```

Output:

Dataset shape: (2240, 29)

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...
0	5524	1957	Graduation	Single	58138.0	0	0	04-09-2012	58	635	...
1	2174	1954	Graduation	Single	46344.0	1	1	08-03-2014	38	11	...
2	4141	1965	Graduation	Together	71613.0	0	0	21-08-2013	26	426	...
3	6182	1984	Graduation	Together	26646.0	1	0	10-02-2014	26	11	...
4	5324	1981	PhD	Married	58293.0	1	0	19-01-2014	94	173	...

5 rows × 29 columns

2. Initial Exploration

```
#Check dataset basic info
data.info()

#Get summary statistics
data.describe(include='all')
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                    2240 non-null   int64
1   Year_Birth            2240 non-null   int64
2   Education              2240 non-null   object
3   Marital_Status        2240 non-null   object
4   Income                2216 non-null   float64
5   Kidhome               2240 non-null   int64
6   Teenhome              2240 non-null   int64
7   Dt_Customer           2240 non-null   object
8   Recency               2240 non-null   int64
9   MntWines              2240 non-null   int64
10  MntFruits              2240 non-null   int64
11  MntMeatProducts        2240 non-null   int64
12  MntFishProducts        2240 non-null   int64
13  MntSweetProducts       2240 non-null   int64
14  MntGoldProds           2240 non-null   int64
15  NumDealsPurchases      2240 non-null   int64
16  NumWebPurchases        2240 non-null   int64
17  NumCatalogPurchases    2240 non-null   int64
18  NumStorePurchases      2240 non-null   int64
19  NumWebVisitsMonth       2240 non-null   int64
20  AcceptedCmp3           2240 non-null   int64
21  AcceptedCmp4           2240 non-null   int64
22  AcceptedCmp5           2240 non-null   int64
23  AcceptedCmp1           2240 non-null   int64
24  AcceptedCmp2           2240 non-null   int64
25  Complain               2240 non-null   int64
26  Z_CostContact          2240 non-null   int64
27  Z_Revenue              2240 non-null   int64
28  Response               2240 non-null   int64
dtypes: float64(1), int64(25), object(3)
memory usage: 507.6+ KB
```

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...
count	2240.000000	2240.000000	2240	2240	2216.000000	2240.000000	2240.000000	2240	2240.000000	2240.000000	...
unique	NaN	NaN	5	8	NaN	NaN	NaN	663	NaN	NaN	...
top	NaN	NaN	Graduation	Married	NaN	NaN	NaN	31-08-2012	NaN	NaN	...
freq	NaN	NaN	1127	864	NaN	NaN	NaN	12	NaN	NaN	...
mean	5592.159821	1968.805804	NaN	NaN	52247.251354	0.444196	0.506250	NaN	49.109375	303.935714	...
std	3246.662198	11.984069	NaN	NaN	25173.076661	0.538398	0.544538	NaN	28.962453	336.597393	...
min	0.000000	1893.000000	NaN	NaN	1730.000000	0.000000	0.000000	NaN	0.000000	0.000000	...
25%	2828.250000	1959.000000	NaN	NaN	35303.000000	0.000000	0.000000	NaN	24.000000	23.750000	...
50%	5458.500000	1970.000000	NaN	NaN	51381.500000	0.000000	0.000000	NaN	49.000000	173.500000	...
75%	8427.750000	1977.000000	NaN	NaN	68522.000000	1.000000	1.000000	NaN	74.000000	504.250000	...
max	11191.000000	1996.000000	NaN	NaN	66666.000000	2.000000	2.000000	NaN	99.000000	1493.000000	...

11 rows × 29 columns

3. Check and handle Missing Value

```
# Check how many missing values exist in each column
missing = data.isnull().sum()
print("Missing values:", missing[missing > 0])
```

Output:

```
Missing values: Income      24
dtype: int64
```

```
#There is only 'Income' with missing values
#Drop rows with missing 'Income'
data = data.dropna(subset=['Income'])
```

4. Create attributes that might be more meaningful for clustering

```
#Create 'Age' from 'Year_Birth'
data['Age'] = 2025 - data['Year_Birth']

#Create 'Total_Spending'
data['Total_Spending'] = data[['MntWines', 'MntFruits', 'MntMeatProducts',
                              'MntFishProducts', 'MntSweetProducts', 'MntGoldProds']].sum(axis=1)

# Create 'Family_Size'
data['Family_Size'] = data['Kidhome'] + data['Teenhome']

# Convert 'Dt_Customer' to datetime with dayfirst=True
data['Dt_Customer'] = pd.to_datetime(data['Dt_Customer'], dayfirst=True)

# Create number of days since the customer joined
data['Customer_Since_Days'] = (pd.to_datetime("2025-01-01") - data['Dt_Customer']).dt.days
```

5. Convert Categorical to Numeric

```
# Check data types to find object (categorical) columns
print("Categorical columns:\n", data.select_dtypes(include='object').columns)

# One-hot encoding for categorical columns
data_encoded = pd.get_dummies(data, columns=['Education', 'Marital_Status'], drop_first=True)
```

Output:

```
Categorical columns:
Index(['Education', 'Marital_Status'], dtype='object')
```


6. Select Relevant Features and Normalize

```
from sklearn.preprocessing import StandardScaler

# Select numeric columns to scale
features = ['Income', 'Age', 'Total_Spending', 'Family_Size', 'Customer_Since_Days']

# Combine with encoded categorical features
features += [col for col in data_encoded.columns if col.startswith('Education_') or col.startswith('Marital_Status_')]

# Extract and scale the data
X = data_encoded[features]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

7. Check dataset columns and preview the encoded, normalized dataset

```
print(data_encoded.columns.tolist())

data_encoded.head()
```

Output:

```
['ID', 'Year_Birth', 'Income', 'Kidhome', 'Teenhome', 'Dt_Customer', 'Recency', 'MntWines', 'MntFruits', 'MntMeatProducts', 'MntFishProducts', 'MntSweetP
roducts', 'MntGoldProds', 'NumDealsPurchases', 'NumWebPurchases', 'NumCatalogPurchases', 'NumStorePurchases', 'NumWebVisitsMonth', 'AcceptedCmp3', 'Accep
tedCmp4', 'AcceptedCmp5', 'AcceptedCmp1', 'AcceptedCmp2', 'Complain', 'Z_CostContact', 'Z_Revenue', 'Response', 'Age', 'Total_Spending', 'Family_Size',
'Customer_Since_Days', 'Education_Basic', 'Education_Graduation', 'Education_Master', 'Education_PhD', 'Marital_Status_Alone', 'Marital_Status_Divorced',
'Marital_Status_Married', 'Marital_Status_Single', 'Marital_Status_Together', 'Marital_Status_Widow', 'Marital_Status_YOLO']
```

	ID	Year_Birth	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	MntFruits	MntMeatProducts	...	Education_Graduation	Education_Master	Educat
0	5524	1957	58138.0	0	0	2012-09-04	58	635	88	546	...	True	False	
1	2174	1954	46344.0	1	1	2014-03-08	38	11	1	6	...	True	False	
2	4141	1965	71613.0	0	0	2013-08-21	26	426	49	127	...	True	False	
3	6182	1984	26646.0	1	0	2014-02-10	26	11	4	20	...	True	False	
4	5324	1981	58293.0	1	0	2014-01-19	94	173	43	118	...	False	False	

5 rows × 42 columns

3.3 Results of Data Preparation

Firstly, we began by loading the customer personality dataset and inspecting it for missing values. We handled the missing data by dropping records with null 'income' values to preserve data integrity. Next, we performed feature engineering by creating new meaningful variables such as Age (derived from birth year), Total_Spending (the sum of all product expenditures), Family_Size (sum of kids and teenagers at home), and Customer_Since_Days (how long the customer has been with the company). The Dt_Customer column was properly converted to datetime format with the correct dayfirst setting to avoid conversion errors.

After engineering new features, we applied encoding to transform categorical variables (Education and Marital_Status) into numerical format for modelling. Finally, we standardized all numerical features using 'StandardScaler' to normalize the data, making it suitable for clustering and dimensionality reduction techniques. As a result, we now have a fully cleaned and numerically prepared dataset (X_scaled) ready for unsupervised learning analysis such as PCA and clustering.

4.0 Model Development

4.1 Overview of Unsupervised Learning Methods

In unsupervised learning, we use clustering dataset that is partitioning a set of data into set of classes, called clusters that members of each class will share some interesting common properties. In this project, we aim to discover hidden customer segments based on demographic and their behavioural such as age, income, spending habits, and family size. We select two widely used clustering algorithms: K-Means Clustering and Hierarchical Clustering, due to their effectiveness in market segmentation and customer profiling tasks.

- **K-Means Clustering:** It is a partitioning algorithm that divides the dataset into a predefined number of clusters by minimizing intra-cluster variance. Customers are grouped based on similarity in their numerical attributes, making it suitable for large, standardized datasets like ours. We used the Elbow Method to determine the optimal number of clusters.
- **Hierarchical Clustering:** This technique builds a tree-like structure to illustrate how data points can be merged or split into clusters. It does not require a predefined number of clusters and is useful for visualizing the nested structure of customer relationships.

By applying both K-Means and Hierarchical Clustering, we can compare a centroid-based partitioning method with a connectivity-based hierarchical method, allowing us to gain deeper insights into customer segmentation and the structure of purchasing behaviour.

Partitioning Methods – K-Means algorithm

First of all, to determine the appropriate number of clusters (k) for our dataset, we applied the Elbow Method, which is a graphical technique used in K-Means clustering. This method involves plotting the Within-Cluster Sum of Squares (WCSS) against different values of k (number of clusters). The goal is to identify the point at which adding more clusters yields diminishing returns, the point is referred to as the "elbow point".

We ran K-Means clustering for values of k ranging from 2 to 11 (dataset has only ~2200 rows 2-11 is well-fit) and plotted the corresponding WCSS values. The resulting Elbow plot showed a noticeable inflection at $k = 7$, indicating that seven clusters balance both compactness and simplicity. Therefore, we selected $k = 7$ as the optimal number of clusters for our K-Means model.

```

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

#Set a consistent graph style
sns.set(style="whitegrid")

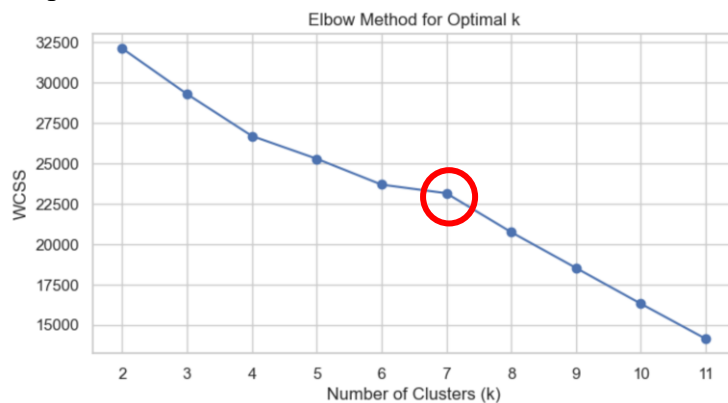
#Use Elbow Method to choose optimal number of clusters (k)
wcss = [] # Within-cluster sum of squares
K_range = range(2, 12)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the Elbow graph
plt.figure(figsize=(8, 4))
plt.plot(K_range, wcss, marker='o')
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")
plt.xticks(K_range)
plt.show()

```

Graph



output:

With the optimal k determined, we applied K-Means clustering with $k = 7$ to the scaled dataset. The features used for clustering included Demographic variables (e.g., Age, Income, Family_Size), Behavioral metrics (e.g., Total_Spending, Recency), Duration of relationship (Customer_Since_Days) and One-hot encoded categorical variables (Education, Marital_Status)

The dataset was standardized using StandardScaler to ensure that all features contributed equally to the clustering.

Each data point (customer) was assigned a cluster label ranging from 0 to 6

- **Cluster 0:** Younger customers with lower income and minimal spending.
- **Cluster 1:** Customers with medium income and high gold and wine product spending.

- **Cluster 2:** Higher income group with above-average spending across most product categories.
- **Cluster 3:** Families with more children and lower spending behavior.
- **Cluster 4:** Long-term customers with average income and moderate engagement.
- **Cluster 5:** Recently joined customers with low recency values and high online activity.
- **Cluster 6:** High-income, loyal customers with high overall engagement and spending.

Each cluster represents a distinct customer segment, which can be used for targeted marketing, customer retention strategies, or personalized recommendations.

```
#Set plotting style
sns.set(style="whitegrid")

#Apply K-Means with the chosen number of clusters - 7
k = 7
kmeans_final = KMeans(n_clusters=k, random_state=42)
cluster_labels = kmeans_final.fit_predict(X_scaled)

#Add cluster labels to DataFrame
data_encoded['Cluster'] = cluster_labels

#View sample data with cluster labels
print(data_encoded[['Income', 'Age', 'Total_Spending', 'Cluster']].head())
```

Output:

	Income	Age	Total_Spending	Cluster
0	58138.0	68	1617	0
1	46344.0	71	27	0
2	71613.0	60	776	2
3	26646.0	41	53	2
4	58293.0	44	422	1

Hierarchical Clustering

First of all, to determine the appropriate number of clusters for our dataset using Hierarchical Clustering, we employed a dendrogram, a visual representation of the clustering process. Hierarchical clustering does not require predefining the number of clusters. Instead, it builds a tree-like structure based on the similarity. It starts with each point as a cluster and merges them from bottom to up step-by-step.

We used Ward's linkage method which aims to minimize the variance within each cluster during the merging process. By generating the dendrogram, we observed the vertical distances between cluster merges to identify natural breaks in the data.

The dendrogram revealed a clear jump in merge distances between 5 and 6 clusters, suggesting that cutting the dendrogram at 5 clusters offers a meaningful segmentation. This cutoff reflects the point before clusters become too broad and start merging dissimilar data points. Based on

this analysis, we selected 5 clusters as the most suitable number for our Hierarchical Clustering model.

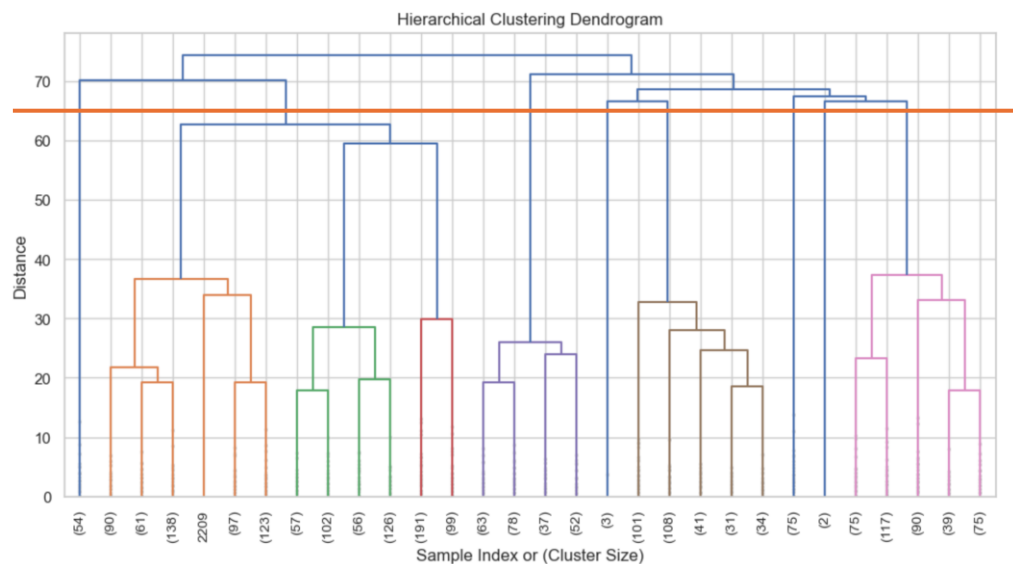
1. Data preprocessing and apply agglomerative(bottom-up) hierarchical Clustering

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Compute Linkage matrix (using Ward's method)
linked = linkage(X_scaled, method='ward') # Ward minimizes within-cluster variance

plt.figure(figsize=(12, 6))
dendrogram(linked, truncate_mode='lastp', p=30, leaf_rotation=90., leaf_font_size=10., show_contracted=True)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Sample Index or (Cluster Size)')
plt.ylabel('Distance')
plt.show()
```

Output:



2. Determining the Optimal Number of Clusters

```
#On the dendrogram, draw a horizontal line at height = 65(big step in merge height).
#Count how many vertical lines it crosses.
#That's your optimal number of clusters.
num_clusters = 5
cluster_labels_hier = fcluster(linked, num_clusters, criterion='maxclust')

# Add Labels back to the DataFrame
data_encoded['Hier_Cluster'] = cluster_labels_hier
```

3. Cluster Profiling

Provide a cluster profiling based on our cluster that helped us interpret the nature of each cluster.

```
# Example cluster profiling
exp_profile = data_encoded.groupby('Cluster')[['Income', 'Age', 'Total_Spending', 'Family_Size']].mean()
print(exp_profile)
```

Output:

	Income	Age	Total_Spending	Family_Size
Cluster				
1	20306.259259	47.537037	81.796296	0.722222
2	51869.317266	54.163891	607.992112	0.914110
3	53092.839130	58.478261	615.530435	1.004348
4	52586.490566	57.562893	607.713836	1.000000
5	56166.221987	59.980973	660.291755	0.989429

5.0 Model Performance and Evaluation

5.1 Evaluation Metrics for Unsupervised Learning

In unsupervised learning, there are no predefined class labels to compare with. Therefore, traditional classification metrics such as accuracy, precision, recall, and F1-score are not suitable for evaluating clustering results. Instead, clustering performance is assessed using internal validation metrics that measure compactness and separation of clusters.

For this project, the clustering evaluation metrics used are:

1. Silhouette Coefficient: Measures how well each data point fits within its cluster. It measures the cohesion and separation.
 - Score ranges from -1 (wrong clustering) to $+1$ (ideal clustering)
2. Elbow Method (for K-Means only): Helps determine the optimal number of clusters using WCSS (Within-Cluster Sum of Squares)

5.2 Analyze Performance

To evaluate the performance of the K-Means and Hierarchical clustering, we used Silhouette Score to measure the cohesion and separation of clusters, and the Elbow Method to determine the optimal number of clusters in K-Means. These quantitative metrics were complemented by qualitative evaluation based on the interpretability and business usefulness of each cluster.

5.2.1 K-Means Clustering

– Silhouette Score

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Fit KMeans with k = 7
kmeans = KMeans(n_clusters=7, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Calculate Silhouette Score
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
print("K-Means Silhouette Score:", round(kmeans_silhouette, 4))

```

K-Means Silhouette Score: 0.2092

The Silhouette Score for K-Means was 0.2092, which indicates that the clusters are moderately well-separated to become 7, though not highly distinct. This is typical in real-world datasets where customer behaviours overlap and boundaries between groups are less defined.

Despite the modest score, the clustering results were meaningful. Through K-means, we identified 7 customer segments, each with distinct characteristics in terms of income, product spending, recency, and engagement channels. These clusters enable practical actions such as:

- Prioritising loyalty rewards for high-income, high-spending customers,
- Designing onboarding campaigns for recent online users, or
- Offering price-based promotions to low-budget and young customers.

These clusters enable more personalized marketing strategies to be promoted.

Likewise, it is important to note that the Elbow Method is only applicable to K-Means clustering, as it requires a predefined number of clusters (k) to be set before training. The Elbow Method helps identify the optimal k by evaluating how the within-cluster variance (WCSS) changes as k increases. It showed an inflection point at k = 7, deciding the chosen number of clusters for K-Means.

5.2.2 Hierarchical Clustering - Silhouette Score

```

from sklearn.cluster import AgglomerativeClustering

# Fit Agglomerative Clustering with k = 5
hierarchical = AgglomerativeClustering(n_clusters=5)
hierarchical_labels = hierarchical.fit_predict(X_scaled)

# Calculate Silhouette Score
hier_silhouette = silhouette_score(X_scaled, hierarchical_labels)
print("Hierarchical Clustering Silhouette Score:", round(hier_silhouette, 4))

```

Hierarchical Clustering Silhouette Score: 0.2168

In contrast, Hierarchical Clustering does not require specifying k in advance, as it builds a hierarchy of clusters from the data. The number of clusters has been chosen by analyzing the dendrogram rather than WCSS, which is why Elbow Method is not used for evaluating.

Hierarchical Clustering achieved a slightly higher Silhouette Score of 0.2168, indicating better intra-cluster cohesion compared to K-Means. This suggests that the clusters formed were slightly more internally consistent and better separated from each other.

In addition to the numerical score, Hierarchical Clustering offered strong visual interpretability through the dendrogram, which allowed us to observe how customers were merged to different levels. Based on the dendrogram, the dataset was segmented into 5 clusters. These clusters are especially helpful for strategic-level segmentation, such as designing customer personas or structured tiered loyalty programs.

The computed evaluation results for both clustering models are summarized below:

Model	Number of Clusters	Silhouette Score
K-Means Clustering	7	0.2092
Hierarchical Clustering	5	0.2168

Although Hierarchical Clustering achieved a slightly higher Silhouette Score, K-Means provided more detailed segmentation, making it more appropriate for use cases such as targeted advertising or tailored promotions. In practice, both models complement each other by offering a blend of strategic overview and tactical customer-level insights.

6.0 Conclusion

In conclusion, this project successfully explored the application of unsupervised learning techniques, specifically K-Means and Hierarchical Clustering, to identify meaningful customer segments from Customer Personality Analysis dataset. Through careful data preparation, feature engineering and clustering, we uncovered hidden patterns in customer demographics, spending behaviours, and engagement history.

K-Means clustering, guided by the Elbow Method, resulted in 7 distinct clusters, each reflecting unique customer characteristics. Hierarchical Clustering, interpreted through a dendrogram, produced 5 broader customer segments with slightly better internal cohesion, as indicated by the higher Silhouette Score. The findings enable businesses to better understand their customer base and implement data-driven strategies.

Overall, the project demonstrated the power of clustering in enabling personalized marketing, improving customer engagement, and enhancing decision-making processes. Future improvements could include incorporating more behavioural data or applying dimensionality reduction techniques like PCA for better visualization and performance.